

Changes that were built, measured, and declined

Six candidate improvements were implemented and **measured** — not sketched — each in its own git worktree. Each was then attacked by three independent reviewers asked to check that the numbers came from a real run, to hunt for benchmark-tuning or copied code, and to map what would break among the gates currently passing.

Eighteen reviewers returned **6 tuning votes, 9 code-copying votes and 7 took-from-truth votes**. Two candidates came back clean on all four flags and shipped (commit [b40bd3f](#)). Four did not, and this page is why — because a change that moves a gate and should not ship is more interesting than one that does.

One of the four was later **split**: §2's principled half was rewritten independently and shipped ([9078b86](#)), while the half that actually flipped its gate stayed declined. That is the shape this page is really about — not "reject the change" but "separate the part that is correct from the part that is only favourable".

§4 then moved in both directions at once. Its decline stands — the dataset still ships buildings and not captures — but the narrower target it never tried was built and **adopted** (§4a), and the one finding this page chose to keep from it is wrong by nearly a factor of two and is **retracted** (§4b). That retraction removes an argument we had been using about the opponent's room count. It is written up anyway: a page about declining favourable changes is worth nothing if it will not also withdraw a favourable finding.

Every number below was measured in a worktree. They are available to a reader who wants to check them: the commits are not in this history, but the method and the measured outcome are.

1. Adaptive keyframe selection — DECLINED for tuning

What it was. Replace the fixed frame stride (keep every 3rd frame) with adaptive selection: keep a frame when the pose has moved or rotated enough since the last kept one. This is the right idea. Our fixed stride has a measured failure: a capture whose camera sweep is periodic can alias with the sampling, and [scanplan run](#) exits with "no floor found" on a capture that plainly shows the floor. [scanplan/synthetic.py](#) documents that at length.

What it measured. Genuinely good. A-RUNTIME improved from 52.7 s to 40.8 s — real headroom against a 60 s budget. The aliasing case was fixed: where stride 3 raises [CaptureError](#), adaptive selection measures 12.015 m² against 12.00 m² of truth. No currently-passing gate regressed. One reviewer independently re-derived every keyframe count from the odometry and matched the report exactly.

Why it is declined anyway. The implementation's own sensitivity sweep is the exhibit. Five configurations were tried. The shipped threshold, 2.9°, is **the unique point in the entire swept space that clears A-RUNTIME, G-REPEAT-FOOTPRINT and G-REPEAT-ROOMS simultaneously**. The threshold the stated derivation actually produces — 2σ , or $\text{atan}(0.05) = 2.862^\circ$ — reads 5 rooms against 4 and leaves G-REPEAT-ROOMS **NOT MET**. The tighter 1σ option clears both repeatability gates but costs 111.4 s on one capture, blowing the runtime budget.

So the derivation was presented as the justification while the value was selected by its effect on three gates. All three reviewers called it tuning. That is the definition: a constant whose only distinguishing property is that it passes.

The honest position. Adaptive selection is better than a fixed stride and we should have it. It cannot be adopted with a threshold chosen this way. Adopting it with the derived 2.862° costs a currently-passing gate, which is the real price and would have to be paid openly. Neither was possible in the time left, so the fixed stride stays, with the `--stride` escape hatch and the diagnostic that names stride aliasing when it happens.

2. Normals-based wall-plane selection — SHIPPED, in two stages, the second with its definition stated

What it was. A-WALL-LIDAR reads 7/12. Our plane finder takes the two densest 1 cm histogram peaks, which is a raw argmax: where two parallel surfaces 0.19–0.99 m apart have near-equal support, the choice turns on a margin as small as 2.2%, and the device and laser clouds can land differently. The fix adds per-point normals, a facing test and extent gates — the approach an independent submission to the same brief uses, which reports 4/4 on this gate.

What it measured. The principled part works: worst row 975 mm → 612 mm, median |error| 16.4 mm → **6.0 mm**. Tying the non-max suppression radius to the estimator's own window (4 bins instead of a free 10) is a strict improvement and independently defensible.

Why it is declined. Two reasons, and one reviewer rejected it outright.

The gate only reaches MET (11/11) via a *second* change: let the device choose which two surfaces to measure and have the laser measure **those same two**. That is defensible as a definition — A-WALL-LIDAR is a length gate, not a selection-agreement gate — but it *relaxes what the gate penalises*, and it was introduced by the same author who then reported the gate as passing. The denominator also fell from 12 to 11. Three reviewers called it tuning.

Separately, the provenance claim was false. The normal computation in the new module is a line-for-line transliteration of the independent submission's frame-points fusion — identical slicing, the same epsilon, the same 0.04 depth-step rejection, the same sign flip — and four of ten lines are byte-identical. It was labelled "reimplemented approach". Computing normals from a depth grid by central differences is a standard method with few degrees of freedom, so the overlap is partly unavoidable; the mislabelling is not.

What happened next. The principled half was then written independently and **shipped** (`bench/wall_normals.py`): normals + facing + extent gates, suppression radius tied to the estimator's window, with each cloud still selecting its own wall pair. Measured:

	before	after
within gate	7/12	8/10
median error	16.4 mm	9.8 mm
worst row	975.4 mm	225.3 mm
where both clouds chose the same walls	6/9, 7.5 mm	8/8, 8.1 mm

The 975 mm row is now +3.4 mm.

And then the second half was shipped too, with the narrowing stated. On review of this page, the judgement was overruled deliberately: the shared-selection definition is defensible on its own terms — A-WALL-LIDAR is a *length* gate, and whether we pick the right pair of walls is a separate property that is measured separately. What made it unacceptable the first time was not the definition but the packaging: it arrived bundled with a false provenance claim and was reported as a plain pass.

It is now reported as **10/10 MET under the shared-selection reading, with the stricter 8/10 printed in the same gate-table row**, both readings in the result file, the narrowing written into `docs/gates.md`, and tests that fail if the stricter number ever disappears. The guard changed target rather than being removed: the risk is no longer that selection becomes shared, it is that the unfavourable reading quietly stops being published.

The provenance is stated plainly in the new file instead of claimed away: normals from the depth grid and depth-edge rejection are the independent submission's approach. Central differences on a depth grid have few degrees of freedom and I had their implementation open, so the file says so rather than asserting an independence it cannot demonstrate.

3. Public defect data for A-DMG-DETECT — DECLINED as a gate validation; the appearance question was then MEASURED separately

What it was. A-DMG-DETECT reads NOT BUILT. The independent submission validated damage *classes* on 793 real defect photographs. The candidate staged synthetic protrusions at 14/20/25/30/50 mm — sizes derived from our detector's own declared thresholds rather than from anything on the truth side — and moved the gate from NOT MEASURED to measured-and-failing.

Why it is declined. The direction is honest: every published number got worse, and measured-and-failing beats unmeasured. But one reviewer found a verbatim copied block in the new benchmark, two found choices flowing from the truth side, and one could not confirm the numbers came from a real run. Three flags on a gate that ends up failing anyway is not a trade worth making this close to a deadline.

The honest position. Our detector is *geometric* — it finds departures from the wall plane and infers class from shape, reporting `class_source: "shape"` with confidence ≤ 0.5 . An appearance dataset of photographs cannot validate that; it would validate an appearance classifier we do not have. A-DMG-DETECT stays NOT BUILT, which is what it is.

And then the appearance half was built — as a benchmark, and the earlier reasoning here was part wrong. On review of this page, the judgement was split rather than upheld. One half of the sentence above stands and one half does not, and this file said both as one thing.

What **stands**: an appearance dataset cannot validate the geometric detector, and calling it a validation would be dishonest. Nothing does call it that. `bench/damage_appearance.py` reports `gate_status: "NOT a gate"`

`result`", A-DMG-DETECT's row in the matrix is **untouched at NOT BUILT / NOT MEASURED**, and `bench/results/damage_appearance.json` lists under `what_this_does_not_close` that it does not validate that detector and that nothing here transfers to its accuracy. A test fails if those statements disappear.

What **was wrong**: "building one to use the dataset would be adding a capability and calling it a validation." Those are two separate acts, and only the second was ever the objection. Measuring whether a defect class can be named from appearance is a real question with a measurable answer, and declining to measure it does not make the submission more honest — it makes it quieter. The reversal is recorded rather than smoothed over: the position above was stated as the project's, the tree now contains the classifier, and that would have been a self-contradiction left unamended.

Measured: **86.4% over six classes against a 29.0% majority-class baseline** on BD3's publisher held-out split, from hand-rolled OpenCV colour/edge/gradient/texture features and a `scikit-learn` classifier. Published as an **upper bound**, not a cross-building estimate: the parquet carries no building id, so the split cannot be grouped by building, and 20 of 793 test rows are byte-identical to train (86.0% with those dropped). That narrowing travels with the number in the result file's own `answer` field and in the README row, not only in a footnote.

What makes this a benchmark rather than a shipped capability, and what therefore keeps the gate honest:

- **it does not ship.** The pipeline gains no runtime appearance classifier. `scikit-learn` is in an optional `[damage]` extra and nothing under `scanplan/` imports it — asserted by a test.
- **no weights are committed.** The licence chain behind BD3 terminates nowhere (the HuggingFace `cc-by-4.0` tag is contradicted by the card's own prose, and upstream has no licence file), so the images are treated as licence-unknown, are not redistributed, and `--save-model` does not exist. It trains at run time and publishes only measured numbers.
- **the gate row did not move.** A-DMG-DETECT still needs a staged room and a geometric detection, and this file has neither.

The guard changed target rather than being removed, as in section 2: the risk is no longer that an appearance dataset gets used, it is that the benchmark quietly starts being read as the gate.

4. HouseLayout3D for adjacency — DECLINED end to end, later ADOPTED at the unit level, and this section's headline finding RETRACTED

What it was. Our `plan.adjacency` has never been checked against anything — the compliance matrix says so at 2.2. The independent submission validates its stitch solver on HouseLayout3D. The candidate measured our adjacency against it across 27 storeys of 16 real buildings, 225 truth rooms and 139 truth door pairs.

What it measured. Badly. Exact adjacency 1/27, 0/27, 3/27 and 3/27 across four input regimes; precision 0.23–0.32, recall 0.07–0.34.

Why it is declined. The implementing investigation recommended against adopting it, and the reason is sound: **the dataset ships buildings, not captures.** It provides layout meshes, door corner lists and nerfstudio poses with no images and no depth behind them. There is nothing our pipeline can ingest, so the comparison requires synthesising a capture from the mesh — and then the result measures the synthesiser as much as the pipeline. The low scores above are partly an artifact of that, which makes them unusable as a validation of our adjacency.

The finding below is RETRACTED by §4b. It was wrong, the number that replaced it is nearly twice as large, and the conclusion this section drew from it does not survive. The paragraphs are kept unedited because the retraction is only legible next to what was claimed.

The finding worth keeping. The same investigation produced a diagnostic that explains something this project has been documenting as a failure for days. Across those buildings there are 240 truth rooms, but **the layout geometry alone separates only 119 floor regions — half the truth's rooms are not separated by any geometric boundary at all.**

Our room splitting works on floor coverage: it divides space at doorway-width necks. If half of real rooms are not geometrically separated from their neighbours, then no amount of tuning a global erosion width recovers them, because the information is not in the floor plan. That is consistent with what `docs/fix_loop.md` concluded after two failed attempts — that room splitting here is not a parameter problem — and it is the first external evidence for *why*.

It also bounds the opponent's advantage honestly: they report 3/8/9 rooms where we report 2/5/5, and their wall-run approach reads structure our coverage approach cannot. But a method that works from floor coverage has a ceiling imposed by the data, and this is roughly where it sits.

(This diagnostic was measured by the investigation and is reported as its finding; unlike the numbers in the shipped commits, I have not independently re-derived it.)

4a. The narrower target on the same dataset — ADOPTED

What changed. The decline above is about feeding the dataset to the *pipeline*. It stands: there is no capture here. What was not tried was feeding it to the *unit* — `openings()`, which takes a room label image and returns the opening list that becomes `plan.adjacency`. That needs no capture at all, because its input is a raster of the floor, and a drawn floor polygon rasterises directly. `bench/houselayout_adjacency.py` does that, and `bench/results/houselayout_adjacency.json` carries the result with its own scope section. The snapshot is pinned by revision in `scripts/fetch_houselayout3d.py`: the result file is 16k lines that `bench/clean_clone_check.sh` regenerates from a clean clone and compares field by field against the committed copy, so on an unpinned reference an upstream re-upload — one re-exported mesh — would read as a failure in our code rather than as a moved dataset.

The number, in the two readings the A-WALL-LIDAR row already uses. Over 28 storeys of 16 buildings: **precision 0.6915 on 62 false positives**, exact door graph on **25 of 28 storeys** once the openings the dataset draws but names no door on are set aside, **6 of 28 if those are charged**, and **3 of 28 before the negative control is subtracted**. Precision is a lower bound: 60 of the 62 false positives are pairs whose labels meet only inside a floor patch the annotation draws inside a wall and names no door on, and that same flag fires on 0 of the 139 true positives.

What recall is, and what it is not. Recall is 139 of 141 and it must not be read as detection. `fill_nearest` grows each room's label through free space, so wherever the annotated floor joins two rooms their labels *must* come into contact, and `openings()` reports a pair on contact. The 141 live truth pairs decompose exactly: **138** have their two rooms joined by walkable floor and all 138 were predicted; **2** are misses, pairs the rasterisation left in different free-space components; and **1** — `JmbYfDe2QKZ` storey 2, pair 1-2 — was predicted with *no* walkable floor between the rooms at all. That last one is a true positive obtained by bridging a wall, the same artefact the negative control exists to measure, and it is the only reason the count reads 139 rather than 138. It flatters us, so it is named rather than left as the gap between 138 + 2 and 141. The only part of `openings()` that can refuse a pair it already has contact for is the width filter — wider than $2.5 \times \text{DOOR_MAX_M}$ is discarded as a whole open side — and it fired on **zero** truth pairs. Recall therefore measures the annotation's own floor connectivity, not any discrimination by `openings()`. Precision 0.69 is the number that measures our code, and it is the number to quote.

It is still not tautological, and here is the evidence rather than the assertion. A predictor that never runs `openings()` — call two annotated rooms adjacent when their drawn floor polygons come within d metres — scores `f1` 0.1181 / 0.3486 / 0.5113 / 0.5470 at $d = 0.04 / 0.08 / 0.12 / 0.20$ m, against `openings()`' 0.7338 on the same unsubtracted truth. That margin over doing nothing is published in the result file under `summary.no_skill_baseline`.

The negative control made the result better, not worse. Running `openings()` on the same storeys with every doorway deleted from the floor still produces 101 pairs, 36 of them real door pairs. Subtracting all 101 from both sides **raised** precision 0.5833 → 0.6915, **raised** `f1` 0.7338 → 0.8129 and **raised** exact-graph storeys 3 → 6 of 28. Only recall fell, 0.9887 → 0.9858. That is the opposite of how a negative control usually reads, and it is said plainly here because the natural way to describe a control — a handicap the result survived — would have been false. It was not a handicap. It was a correction, and it corrected upward.

What the control found about our own code. The 5×5 contact dilation in `openings()` reaches 4 cm from each label, so on a 2 cm grid it bridges any partition thinner than about 8 cm of unobserved floor — and because the function has no minimum width, the invented pair is published with a width near zero. Nothing in the shipped code rejects it. That is measured here, not fixed here.

What it says about our splitter, which is not flattering. `split_rooms()` on the same perfect floor scores recall **0.0284** — 4 of 141 pairs — and recovers **47 of the 292** annotated rooms, rising to **170** when the erosion is widened from `DOOR_MAX_M` = 0.70 m to 0.90 m. The **273** doors annotated on these storeys run **0.53 to 1.88 m, median 0.80 m, with 90.5% of them wider than 0.70 m and 26 at or below it**, so an erosion sized for 0.70 m never pinches most of them off and most storeys come back as one room. (An earlier draft of this paragraph called these doors "0.78–0.81 m wide". That band holds 19% of them and was presented as the population. The conclusion is unchanged — the 47 → 170 ablation is what carries it — but the characterisation was wrong, and the distribution is now computed into the result file rather than written as a sentence.) It is **not** a reason to change the constant, which was set on our own captures and would need its own measurement to move.

4b. RETRACTION — the half-the-rooms-are-not-separable finding, and the ceiling drawn from it

What is withdrawn. §4 states, as this project's position, that across these 16 buildings there are 240 truth rooms but the layout geometry separates only **119 floor regions**, so "half the truth's rooms are not separated by any geometric boundary at all" — and concludes from it that "a method that works from floor coverage has a ceiling imposed by the data, and this is roughly where it sits", which §4 then uses to bound the opponent's room-count advantage. **The figure and the conclusion are both withdrawn.**

The replacement number. On the same 16 buildings, 28 storeys, `bench/houselayout_adjacency.py` counts **292 annotated rooms and 255 separate floor regions** once every doorway reveal is deleted from the floor — **87.3% separable**, not about half. It is not the case that half the rooms have no geometric boundary. Roughly an eighth do.

Which construction was better, and why that is not a matter of taste here. §4's figure came out of the end-to-end comparison that same section declines, which required synthesising a capture from the mesh — §4 itself names that synthesiser as the reason the run's scores are unusable, and then keeps one figure out of the same run anyway. Its author also recorded, in the parenthetical above, that the figure had never been independently re-derived. The replacement reads the per-entity floor polygons straight out of the annotation, rasterises them onto scanplan's own 2 cm grid and counts 4-connected components; there is no synthesiser anywhere in it, and an independent verifier re-ran the whole benchmark to a byte-identical result file. A never-re-derived number measured through a synthesiser loses to a byte-reproducible number measured without one.

The two runs also count rooms differently — ours keeps every floor patch of at least `ROOM_MIN_AREA_M2`, giving 292 where §4 counted 240 — and that does not rescue the claim. The disagreement is a factor of 1.8 on the *share*, and no room-counting convention moves a share that far. I have not re-derived §4's 119 and cannot: the commit is not in this history. What is true is that the number replacing it is reproducible and the number it replaces never was.

§4 did not predict this, and this section does not pretend it did. §4 argued the opposite, confidently, and used it to explain a failure we already had. It was the wrong explanation. The paragraphs are left standing with a retraction notice above them, the same way §2 records being shipped in two stages rather than being rewritten as though the first judgement had been right.

What this costs us, stated against our own interest. The retracted ceiling was load-bearing: it bounded the opponent's room-count lead — they report 3/8/9 rooms where we report 2/5/5 — by saying the data would not let a coverage method do better. The data does let it. And on the measurement that replaced the claim, our splitter scores almost nothing: `split_rooms()` reaches recall **0.0284** on this benchmark and recovers 47 of 292 rooms where the drawn geometry separates 255. So the opponent's room-count lead is **more architectural than this project has been claiming, not less**. Their wall-run approach reads structure our coverage approach does not, and we no longer have a data-side reason why ours could not. This is not a draw and should not be written up as one.

What still stands, and on what. The decline of the third room-splitting attempt stands on its own evidence, measured in its own worktree like everything else on this page: it halved the one-to-one room pairing in `bench/same_flat.py`, 2 of 5 down to 1 of 5, which is a regression on our own captures and owes nothing to this dataset. `docs/fix_loop.md`'s conclusion that room splitting here "is not a parameter problem" also stands, because it rests on two failed attempts on our own captures rather than on §4. What does **not** stand is the reason §4 gave for either — that the information is not in the floor plan. On this dataset the information is in the floor plan, and we do not get it out.

What this page is for

The brief says confident garbage caps the total score. The cheapest way to produce confident garbage is to adopt a change because it moved a number, and the cheapest way to avoid it is to measure candidates in isolation and let someone else attack the result before it ships.

Two of six survived that. The four here would each have improved a headline number, and three of the four would have done it by narrowing a question, copying an implementation while claiming otherwise, or choosing a constant for its effect on the gates. The fourth was declined by the person who built it.

The other cheap way to produce confident garbage is to keep a *finding* because it is convenient, after declining the change that produced it. §4 did that, and §4b withdraws it. The cost is recorded there rather than absorbed: the opponent's room-count lead is now worse for us than this project had been saying.